École de technologie supérieure Eric Thé et Hugues Saulnier

INF-155 Travail Pratique #2

La dynamique proie-prédateur Travail en équipes

1 Objectifs

Il y a des situations ou obtenir des données est très difficile, voir impossible. Imaginer de recenser, au cours des années, la population de papillons zébrés sud-africains ou la quantité de mangoustes asiatiques. Pour évaluer de telles populations, on peut à partir de conditions initiales, simuler le comportement d'un écosystème à l'aide d'un automate. Ou bien on peut utiliser certains modèles mathématiques capables de suivre la dynamique des systèmes biologiques. Par exemple les équations différentielles de Lotka-Volterra sont utilisées comme modèle pour représenter les populations des lynx et de lièvres dans le temps. Par contre, par des règles simples on peut aussi simuler le modèle mathématique par un automate.

2 Description du problème : Modèle proie – prédateur

Écrire le programme d'un automate qui simulera un modèle de proies et de prédateurs. Le contexte est l'océan, les proies sont des petits poissons (ti-poissons) et les prédateurs sont des requins. Tous ensemble dans l'océan ils mangent, se reproduisent et meurent.

- La NUTRITION : Les ti-poissons sont herbivores. On va simplifier le modèle en considérant que leur source de nourriture est illimitée. Tous sont alors bien nourris et vous n'avez pas à traiter la faim des ti-poissons. Les requins mangent des ti-poissons et la ressource varie selon la population des poissons. La survie d'un requin dépend du temps passé depuis son dernier repas.
- La REPRODUCTION : La reproduction sera non-sexuée et semblable chez les deux espèces. Dès qu'il aura atteint l'âge de la puberté et sera considéré comme adulte, un animal (poisson ou requin) donnera naissance à un jeune de la même espèce après un temps fixe de gestation. Le jeune deviendra à son tour adulte et va automatiquement procréer s'il survit jusqu'à la puberté. Les temps de gestation et de prépuberté sont normalement beaucoup plus courts chez les ti-poissons que chez les requins. Pour éviter une croissance exponentielle des ti-poissons, on va aussi générer des « fausses couches » avec une probabilité de 33%.
- La MORT : Les ti-poissons meurent sous la dent d'un requin OU par vieillesse. Les poissons vont aussi mourir après leur 3ième accouchement. Les requins meurent d'un jeûne prolongé (trop de jours sans avoir mangé) OU par vieillesse.

3 Modélisation

Nous modélisons la dynamique des populations de poissons et de requins à l'aide d'un automate à états finis. Le système suit un cycle de 4 commandes séquentielles qui représente le passage du temps :

- Identification des poissons morts
- Identification des requins morts
- Mise à jour des états des requins
- Mise à jour des états des poissons
- Recommencer le cycle

Cette approche permet de gérer les interactions écologiques entre les populations en suivant un modèle discret. L'automate est défini par un ensemble d'états, une fonction de transition et une règle d'évolution des individus. Nous définissons deux automates distincts : un pour les poissons et un pour les requins :

Poissons $\boldsymbol{Q_P} = \{q_0$**: Nouveau-né**, q_1 **: Jeune**, q_2**: Adulte**, q_3**: Reproduction**, F_P**: Mort**$\}$

Requins $\boldsymbol{Q_R} = \{q_0$**: Nouveau-né**, q_1 **: Jeune**, q_2**: Adulte**, q_3**: Reproduction**, q_3**: Affamé** , F_R**: Mort**$\}$

4 Modules à faire

On demande la réalisation de 4 modules :

- le module de l'océan : gestion d'une grille d'animaux (poissons et requins) dans une matrice 2D.
- un module de gestion d'animaux génériques : fonctions communes aux poissons ET aux requins.
- le module des poissons ET le module des requins : fonctions spécifiques aux poissons/requins.

Un module de fonctions aléatoires et d'affichages (« util.h », « util.cpp ») vous est donné en plus du module de gestion d'affichages dans la console texte (« winconsole.h », « winconsole.cpp »).

4.1 Module des animaux

Ce module défini le type-enregistrement « t_animal » pour gérer les données spécifiques aux poissons et requins. Chaque animal aura une position (x, y) dans l'océan, un âge (en jours), un niveau d'énergie et le nombre de jours de gestation. Toutes des valeurs entières.

On vous donne le fichier interface « **Animal.h** » de ce module qui contient la liste des fonctions que vous devez écrire dans le fichier « *.cpp » de ce module. Notez que ce sont presque toutes des petites fonctions qui permettent d'accéder aux champs d'un « t_animal » en lecture ou en écriture. Tout ceci dans le bût d'ENCAPSULER le type « t_animal ». Vous devez donc absolument utiliser ces fonctions pour soit lire ou modifier les attributs d'un animal dans tous les autres modules (i.e. poisson, requin) de votre projet.

4.2 Module de l'océan

Le module-océan permet la gestion des poissons et des requins dans l'océan. Il renseigne les poissons et les requins sur l'état des cases voisines dans la matrice de l'océan et coordonne le déplacement de ceux-ci. De plus, il contient la fonction d'affichage à l'écran de l'océan.

Chaque position (x, y) dans l'océan contient deux valeurs :

- Un type de contenu, soit **VIDE, POISSON** ou **REQUIN**
- Un numéro d'identification, soit le numéro du poisson ou requin dans leur liste respective. Sinon, pour une position vide, la valeur de ce numéro est mise à **-1** (**RIEN**)

C'est à vous de définir le fichier-interface « **ocean.h** » de ce module. Ce module doit inclure que le fichier « utils.h ». Utilisez les déclarations de constantes et types suivants :

```
#define HAUTEUR  25
#define LARGEUR  110
#define RIEN     -1

/* Type énuméré pour les 8 directions de déplacement */
typedef enum {HAUT, HAUT_DR, DROITE, BAS_DR, BAS, BAS_G, GAUCHE, HAUT_G} t_dir;

/* Type énuméré pour le contenu d'une case */
typedef enum {VIDE, POISSON, REQUIN} t_contenu;

/* Type structure pour le contenu d'une case de la grille */
typedef struct{
  t_contenu contenu;   //Soit: VIDE, POISSON ou REQUIN
  int       numero;    //le numéro du poisson/requin à cet endroit, sinon -1
} t_case;

/* Type-tableau de la grille de la mer */
typedef t_case t_ocean[HAUTEUR][LARGEUR];
```

À vous de définir les fonctions qui vont agir sur l'océan dans le fichier-implémentation « **ocean.cpp** ». Elles seront toutes déclarées comme des fonctions publiques dans le fichier « ocean.h ».

Vous devez au minimum avoir des fonctions qui vont permettre de :

- **Vider** tout le contenu de l'océan (mettre toutes les cases à {VIDE, RIEN}).
- Obtenir le **contenu** d'une coordonnée (posx, posy).
- Obtenir le **numéro** inscrit à une coordonnée (posx, posy).
- Écrire **un nouveau contenu et numéro** dans une coordonnée (posx, posy).
- **Effacer** le contenu d'une coordonnée (posx, posy).
- Compter **le nombre de cases voisines libres** autour d'une coordonnée (posx, posy).
- **Choisir aléatoirement une case voisine libre** autour d'une coordonnée (posx, posy).
- **Dessiner** l'océan au complet en mode texte (avec les fonction et constantes de « utils.h »).

4.3 Module des poissons

Le module des poissons contient les fonctions nécessaires à maintenir une liste des poissons vivants. Ce module doit inclure le fichier « ocean.h » et le fichier « Animal.h ».

<u>Valeurs initiales des poissons :</u>

Pour définir le comportement d'un poisson, on a besoin des constantes suivantes (dans le fichier *.h) :

```
#define MAX_POISSON         500    //nombre maximal de poissons avant la surpopulation
#define MAX_AGE_POISSON      60    //l'âge maximal d'un poisson
#define NB_JRS_PUB_POISSON   10    //nombre de jours avant la puberté
#define NB_JRS_GEST_POISSON   5    //nombre de jours de gestation
#define ENERGIE_INIT_POISSON  3    //valeur initiale d'énergie d'un bébé-poisson
```

Ensuite pour définir la liste de poissons vous devez utiliser les types suivants :

```
/* Type-structure pour la liste des poissons */
typedef struct{
  t_animal liste[MAX_POISSON];   //tableau statique des poissons
  int      nb_poisson;           //nombre de poissons dans la liste
} t_liste_poissons;
```

À vous de définir les fonctions (8 fonctions publiques, plus 2 sous-fonctions privées) qui vont agir sur un poisson et sur la liste des poissons. Vous devez avoir des fonctions qui vont permettre de :

- Créer et retourner un nouveau poisson avec une position aléatoire. Chaque nouveau poisson devra être généré dans une case VIDE choisie aléatoirement dans l'océan. Les poissons créés auront un âge choisi aléatoirement entre [0, MAX_AGE_POISSON], une énergie de départ de ENERGIE_INIT_POISSON, et 0 jours de gestation (fonction privée).
- Ajouter un poisson à la fin de la liste. Ensuite augmenter le nombre de poissons (fonction privée).
- Vider la liste de poissons (simplement remettre le nombre de poissons de la liste à zéro).
- Créer la liste initiale des poissons. Cette fonction reçoit la liste des poissons, le nombre initial de poissons et l'océan. Pour chaque poisson, créer un nouveau poisson et l'ajouter à la liste. On doit aussi inscrire ce poisson (avec son numéro) dans l'océan à la case où le poisson a été créé.
- Obtenir le nombre actuel de poissons dans la liste.
- Déplacer un poisson de 1 case dans l'océan :

  Cette fonction reçoit la référence au poisson et l'océan. Le poisson ne se déplacera pas s'il n'y a PAS de case libre dans son voisinage immédiat. Notez que dans le sens vertical on va limiter les déplacements, donc un poisson ne sort pas de l'eau et n'ira pas plus bas que le fond de la mer. Par contre, dans le sens horizontal, vous faites traverser le poisson de l'autre côté de l'écran. Donc, un poisson qui dépasse l'extrême-gauche de l'écran va se retrouver à l'extrême-droite et vice-versa. Vous devez faire les modifications nécessaires dans l'océan (vider une case et remplir la nouvelle).
- Retourner le poisson se trouvant à une position donnée `[i]` dans la liste.
- Modifier le poisson se trouvant à une position donnée `[i]` dans la liste (poisson reçu par référence).

- Éliminer un poisson de la liste. On reçoit la liste des poissons, la position du poisson et l'océan. Si la position reçue n'est pas celle du dernier poisson, on remplace la position à vider avec le dernier poisson de la liste (il faudra ensuite remplacer le « *numéro* » de cette case de l'océan). N'oubliez pas de décrémenter ensuite le nombre actuel de poissons et de vider son ancienne position dans l'océan.

- Ajouter un nouveau bébé-poisson à la liste :

  Cette fonction reçoit la liste des poissons, l'océan et une référence au poisson-parent. Le bébé-poisson doit être généré dans une case voisine au poisson-parent. Notez que s'il n'y a pas de case libre autour du poisson-parent, le bébé ne sera pas mis au monde (on attend la prochaine itération). Dans ce cas, on quitte la fonction avec un retour de 0.

  Si le poisson peut procréer on va générer une probabilité de « fausse couche » de 33% (1 chance sur 3), donc dans 66% des cas les poissons vont réellement accoucher. Ces contraintes ainsi que le nombre maximal de poissons dans la liste (MAX_POISSONS) vont permettre d'éviter un scénario de surpopulation de l'espèce-proie dans notre océan. Chaque nouveau poisson ajouté aura son âge initialisé à 0, son énergie de départ à ENERGIE_INIT_POISSON, et aura 0 jours de gestation.

  Si le bébé-poisson est ajouté à la liste, on va réinitialiser le nombre de jours de gestation du poisson-parent à la valeur (-NB_JRS_GEST_POISSON). On doit ensuite ajouter le nouveau poisson à la fin de la liste et inscrire le nouveau poisson dans l'océan. On retourne ensuite 1.

  Si par contre le bébé-poisson n'a PAS été mis au monde dû à une fausse-couche OU dû à la surpopulation on va quand même réinitialiser le nombre de jours de gestation du poisson-parent à la valeur (−NB_JRS_GEST_POISSON) et on retourne 0.

Pour ce module, vous devez écrire le fichier-interface « **poisson.h** » et son fichier-code « **poisson.cpp** ».

4.4 Module des requins

Le module des requins contient les fonctions nécessaires à maintenir une liste des requins vivants. Notez que le code de ce module sera TRÈS semblable à celui du module des poissons. Ce module doit inclure le module « ocean.h » et le fichier « Animal.h ».

<u>Valeurs initiales des requins :</u>

Pour définir le comportement d'un requin, on a besoin des constantes suivantes (dans le fichier *.h) :

```
#define MAX_REQUINS        500    //nombre maximal de requins avant la surpopulation
#define MAX_AGE_REQUIN    1500    //l'âge maximal d'un requin
#define NB_JRS_PUB_REQUIN   90    //nombre de jours avant la puberté
#define NB_JRS_GEST_REQUIN  21    //nombre de jours de gestation
#define JRS_DIGESTION        5    //valeur ajoutée à l'indice d'énergie après un repas
```

Ensuite pour définir la liste de requins vous devez utiliser les types suivants :

```
/* Type-structure pour la liste des requins */
typedef struct{
  t_animal liste[MAX_REQUINS];    //tableau statique des requins
  int      nb_requins;            //nombre de requins dans la liste
} t_liste_requins;
```

À vous de définir les 10 fonctions qui vont agir sur la liste des requins. Ces 10 fonctions seront TRÈS semblables à celles qui gèrent la liste de poissons (presque un copié-collé). Voici les différences notables :

- Créer la liste initiale des requins : Durant la génération de la liste initiale, chaque nouveau requin généré aura un âge aléatoire entre [0, MAX_AGE_REQUINS], une énergie aléatoire entre [JRS_DIGESTION, (JRS_DIGESTION * 4)], et si l'âge aléatoire du requin est supérieur ou égal à NB_JRS_PUB_REQUIN, on va lui attribuer entre [1, NB_JRS_GEST_REQUIN] jours de gestation. Sinon, lui donnera 0 jours de gestation.

- Ajouter un nouveau bébé-requin à la liste : Dans le cas de la reproduction, il n'y aura aucune contrainte de fausse-couche (sauf si la liste est pleine). Chaque nouveau requin aura son âge initialisé à 0, son énergie à (JRS_DIGESTION * 3), et aura 0 jours de gestation. Les bébés-requins sont ajoutés à la fin de la liste et à l'océan et le champ-gestation du parent est remis à (-NB_JRS_GEST_REQUIN).

5 Algorithme de la simulation

Voici, de façon sommaire, l'algorithme général de la simulation de votre modèle (la fonction « *main()* ») :

- On demande le mode de fonctionnement : mode affichage OU écriture dans un fichier (voir étape 7).
- Si l'utilisateur a choisi le mode avec fichier texte, on ouvre le fichier texte des statistiques.
- On établit les conditions initiales (nombre de poissons, nombre de requins, etc.). Commencez avec un ratio de départ d'environ 80% poissons / 20% requins avec l'océan à moitié plein de poissons.
- On vide les structures de données.
- On génère la liste des poissons aléatoirement.
- On génère la liste des requins aléatoirement.
- On affiche l'océan de départ.
- Pour chaque itération (utilisez un compteur « temps » pour représenter le passage des jours) :
    - On incrémente le temps de +1 jour
    - Pour chaque poisson de la liste, on identifie et on supprime les poissons mangés par les requins de la liste (ceux qui sont voisins à un requin dans l'océan). Si un poisson n'a PAS été mangé on vérifie ensuite s'il ne meurt pas de vieillesse.

$$P_{t+1} = P_t - \sum_{i \in P_t} M_i - \sum_{i \in P_t} 1\ (A_i > \text{MAX_AGE_POISSON})$$

M_i : Indicateur si le poisson *i* est mangé par un requin voisin :

$$M_i = \begin{cases} 1 \text{ si le poisson i est voisin d'un requin} \\ 0 \text{ sinon} \end{cases}$$

$1\ (A_i > \text{MAX_AGE_POISSON})$: Fonction indicatrice qui supprime les poissons ayant atteint un âge A_i supérieur à MAX_AGE_POISSON.

Ajouter le montant JRS_DIGESTION au champ d'énergie du requin voisin. Ceci équivaut à donner des forces au requin qui vient juste de manger le poisson

    - Pour chaque requin, on supprime les requins morts de faim OU de vieillesse de la liste.
    - On traite tous les requins (soit on génère un nouveau bébé OU on déplace le requin). Dans les 2 cas on incrémente l'âge (et la gestation) des requins ET on décrémente leur indice d'énergie.
    - On traite tous les poissons (soit on génère un nouveau bébé OU on déplace le poisson). Dans les 2 cas on incrémente l'âge (et la gestation) des poissons. Si le poisson a donné naissance à un nouveau bébé on doit décrémenter son indice d'énergie
    - Si on est en mode affichage,
        - On affiche l'océan et sa population (avec la fonction « afficher_etat() ») et on met le programme en attente avec la fonction « delai_ecran() ».
    - Sinon,
        - On écrit le no. de l'itération, le nombre de poissons et de requins dans le fichier texte.
- On répète la boucle tant qu'il reste des poissons et des requins OU qu'un nombre maximal d'itérations n'est pas atteint (eg. 5000) OU que l'utilisateur n'a pas arrêté la simulation (avec la touche <**ESC**>).
- Suite à la fin de la simulation si on est en mode avec fichier texte, on ferme le fichier.
- On affiche dans la console texte un message qui indique la raison d'arrêt de la simulation (soit : extinction des poissons, extinction des requins ou fin de simulation normale).

Notez que les sous-fonctions principales de cet algorithme, appelées dans la boucle de simulation (eg. « traiter_poissons_morts() », « traiter_requins_morts() », « traiter_poissons() » et « traiter_requins() »), seront déclarées et définies dans le même fichier-source que votre fonction « *main()* ». Ce fichier principal doit inclure que les modules des poissons et des requins.

6 Implantation de tableaux dynamiques pour les deux listes

Une fois que votre programme est totalement fonctionnel, vous devez remplacer les tableaux statiques des types **`t_liste_poissons`** et **`t_liste_requins`** par des tableaux dynamiques. Ceci implique l'ajout dans nouveau champ « **`int`** `taille_liste;` » dans ces deux types struct (ce champ remplacera les constantes `MAX_POISSON` et `MAX_REQUIN` dans le code de ces modules). Faites l'allocation du tableau dynamique dans les fonctions de génération des deux listes initiales. Ceci implique aussi que vous devez ajouter une fonction de nettoyage « liberer_liste() » (pour libérer le tableau) dans vos deux modules.

7 Écriture dans un fichier

Votre programme doit donner deux options à l'utilisateur, soit un mode console ou un mode écriture fichier. Le mode console permet un affichage à l'écran à chaque itération de la matrice de l'océan. Le mode fichier permet l'écriture dans un fichier externe des données de la matrice-océan à chaque itération. Le fichier prendra l'extension « ***.txt** » et pourra être ensuite importé dans Excel pour une analyse future. Cette étape peut être réalisée à la toute fin du projet. Avec un code bien écrit il est très simple d'ajouter l'option d'écriture dans un fichier. Ajouter la fonction qui demande ce mode dans le fichier principal.

8 Rapport à remettre

En plus du code de votre automate, vous devrez remettre un court rapport sur les résultats obtenus en simulation. On cherche à démontrer que notre automate peut se rapprocher des courbes des équations de Lotka-Volterra et du même coup du monde réel. Vous devrez déterminer <u>pour quelle plage de conditions initiales vous obtenez une similitude avec les courbes de Lotka-Volterra</u>. Voici un exemple des courbes de Lotka-Volterra. Le nombre de requins devra être «gonflé» (fois 5) pour bien visualiser sa courbe :

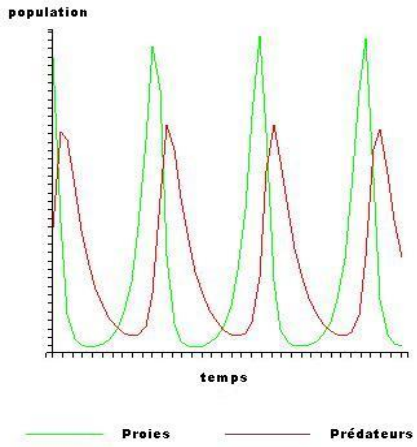

Figure 1 : Exemple de courbes de Lotka-Volterra

Autant pour votre automate que pour les équations, on a besoin de trouver des conditions initiales qui imitent celles d'un monde réel. Pour trouver les bonnes conditions initiales, vous devez exécuter votre automate plusieurs fois avec différentes conditions initiales et ce pour un nombre élevé d'itérations. Les résultats doivent être enregistrés dans différents fichiers textes « .dat » et ensuite analysés dans Excel. Un modèle « stable » devrait rouler pendant au moins 3000 itérations.

<u>Choix des conditions initiales (avec des constantes) :</u>

Les conditions initiales qui vont influencer le comportement de votre modèle sont les suivantes :

- le nombre de poissons de départ;
- le nombre de requins de départ;
- l'âge de la puberté chez le poisson;
- l'âge de la puberté chez le requin;
- le temps de gestation d'un poisson;
- le temps de gestation d'un requin.

9 Structure modulaire du projet

La structure modulaire du projet ainsi que les liens de compilation se feront de la façon suivante :

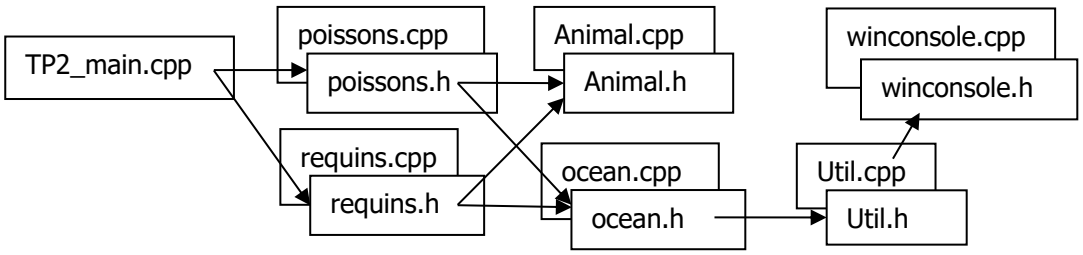

Figure 2 : schéma de la configuration du projet

10 Remise

Votre programme devra respecter les contraintes suivantes :

- Il devra respecter les exigences de remise des travaux pratiques (sur le site Moodle).
- Il ne devra contenir aucune variable globale. Une variable globale entraînera une pénalité de 25 %.
- Il ne devra contenir aucun goto et aucun exit. La présence d'une de ces instructions entraînera la perte de 10 % des points.
- Votre travail devra être remis sur Moodle selon les directives données en classe.

En cas de doute, n'hésitez pas à consulter votre enseignant.

BON TRAVAIL!